Machine Learning - Assignment 2

Jose Pedro Rodrigues*
up202006455

Pedro Leite†
up199904053

Sandra Silva‡
up201403358

ABSTRACT

This project consisted on the development of a classifying model that distinguished text written in Portuguese (Portugal) from Portuguese (Brazil). The process was performed in Google colab with numerous experimentation tests being performed interchanging the model through its hyperparameters, adding data, encompassing word embeddings and using known pre-trained models. The best performing model consisted of using a pre-trainined model called Bertimbau which had an F1-test score of 98%.

KEYWORDS

NLP, PT PT, PT BR, pyTorch, LSTM, Pre-trained components

ACM Reference Format:
Jose Pedro Rodrigues, Pedro Leite, and Sandra Silva. 2024. Machine Learning - Assignment 2. In *Machine Learning, 2024, FCUP*. ACM, New York, NY, USA, 4 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

The geographical and social evolution of languages throughout time is a phenomenon that has led to a significant challenge in machine learning. This hurdle concerns the inclusivity of less prevalent language variants in models that often prioritize language variants that come from regions with larger populations or greater global influence. One popular example is the difference in amount of available content that is written in Portuguese from Brazil in comparison to Portuguese from Portugal. This causes a disadvantage in countries in positions similar to Portugal, in the deployment of language-based models, especially in fields where proper grammar and specific nuances may be crucial.

Our project had the purpose of developing a model capable of classifying the text into each language variant and thus differentiate Portuguese (Portugal) from Portuguese (Brazil). To start our project a notebook in google colab was provided with a head start on a LSTM model. The LSTM (Long Short-Term Memory) RNN model was used with an initial embedding layer corresponding to the encoder (with the size of the vocab) and an output layer with two neurons (classification in PT-PT or PT-BZ). Using the model given by our professors, we decided to modify it in order to enhance it to its best capacity through different strategies. We also noted which changes would contribute the most in the classification of languages. The changes included adding more data to train, and changing the models hyper parameters (mainly through the changing of layer count, hidden size, batch size, epoch count and learning rate). Finally, we incorporated pre-trained components either through word embeddings we found in fasttext or GloVE or through language models such as Albertina or bertimbau. All of these modifications helped us compare each strategy and understand how a conjunction of these methods would affect the categorization process, eventually creating a model that would best classify both language variants. [3]

*All authors contributed equally to this research.
†All authors contributed equally to this research.
‡All authors contributed equally to this research.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Grupo X, May, 2024, FCUP
© 2024 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06
https://doi.org/XXXXXXX.XXXXXXX

2 METHODS AND RESULTS

2.1 The dataset analysis

The original dataset is provided by the HuggingFace platform and named 'cc4051/pt_vid'. It was well balanced (PT-PT and PT-BZ ratio) with 11.7k and 2.57k rows for training and testing, respectively. This test dataset was also considered for the overall model evaluation (i.e., independently of the train dataset considered).

The benefit of using additional datasets was considered and a total of 9 datasets were evaluated from Huggin Face and Github. Only datasets with well-balanced data, restricted to the original label definition and with more than 1k rows were used (Table 1). The LCA PORVID delexicalized_n_grams dataset from HuggingFace, with a considerably larger number of rows was initially considered, but due to a lack of computing power was dropped. Due to a restriction in the colab's gpu utilisation, the number of rows added from the additional datasets were limited to a similar number to the original dataset.

Figure 1 presents the number of tokens for each additional dataset. A different vocab value represents a discrepancy with the original data and, eventually, a lower consistency and quality of the dataset. The web and github datasets have almost 4 time more tokens than the original dataset, hence they should not be used to train the model if the main objective is a high F1 score for the original test (Cc4051/pt_vid).

The analysis involved sequentially adding each dataset to the original (Cc4051/pt_vid), train the model and evaluating the F1 score (using the original test dataset). The DNN had two hidden layer with 200 neuron each, a batch size of 32 and a learning rate equal to 0.001. These hyperparameters are evaluated later. Figure 2 shows the F1 score for training and testing for the previous datasets and reveals that adding the journalist and politics datasets (LCA PORVID – Portuguese vid) guarantees high F1 values. Therefore, if additional data is needed to control the DNN overfitting these should be used.

The importance of each token was evaluated between the additional datasets and the original data (Cc4051/pt_vid). Observing the highly used tokens (with a relative percentage higher than 1%) there was little difference observed between datasets. However, for

Table 1: Dataset Information

Dataset	Subset	Train Balance	Train Rows (k)	Test Balance	Test Rows
Cc4051/pt_vid	-	53.2/46.8	11.7k	51.9/48.1	2.67k
Github DST-TL	-	61.6/38.3	3.5k	59.3/40.7	1.0k
LCA PORVID - frmt	-	50/50	1.97k	-	-
LCA PORVID – portuguese-vid	journalist	81.5/18.5	1.66M	50/50	10k
	legal	99.9/0.1	2.97	50/50	1k
	literature	29.6/70.4	69k	50/50	5k
	politics	89.6/10.4	27.6	50/50	1k
	Social media	99.8/0.2	1.78M	50/50	1k
	web	51.1/48.9	91.1k	50/50	10k

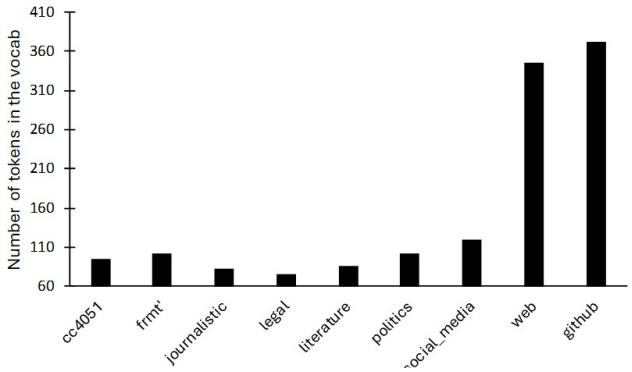

Figure 1: The number of tokens in vocabs in each dataset

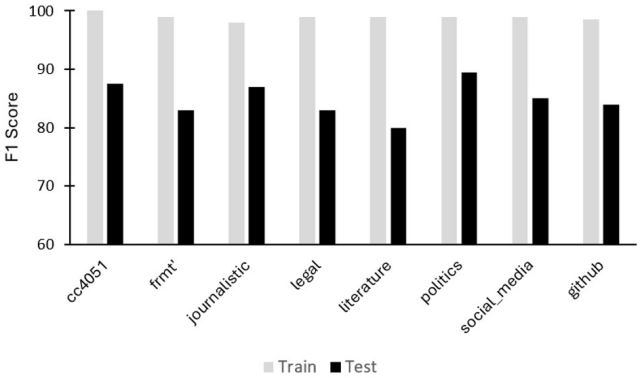

Figure 2: Train and test F1 scores for the dataset evaluated

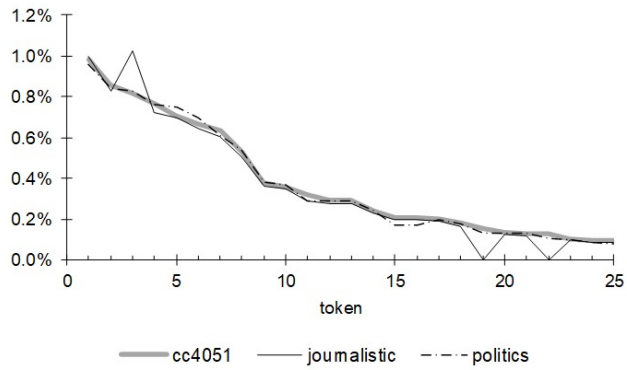

Figure 3: Relative importance of each token (for less than 1%) comparing the cc4051, journalistic and politics datasets

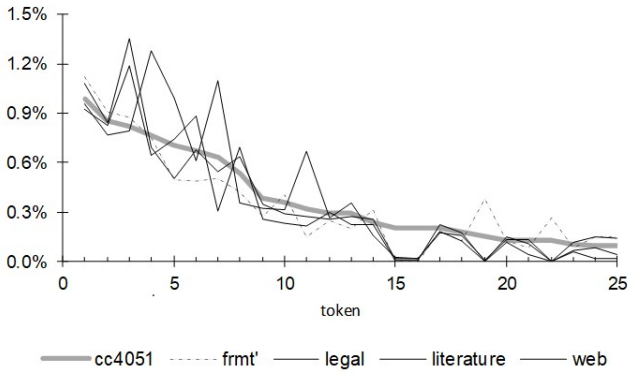

Figure 4: Relative importance of each token (for less than 1%) comparing cc4051, frmt, literature and web datasets

tokens with a lower relative percentage between 1 and 0.1% some differences are observed. The datasets that had similar F1 scores (journalist and politics) to CC4051 present similar trends (Figure 3). However, considering the other datasets (frmt, github,etc...) a higher difference was registered (Figure 4).

2.2 Fine-tuning of the Hyperparameters

2.2.1 *Activation function .*

The *ReLu* and *ELU* activation functions were evaluated, with ELU considered as the default configuration.

2.2.2 *The number of epochs .*

The total number of epochs represents the number of complete passes through the training dataset algorithm. The definition of the number of epochs used is a delicate process that has the objective of helping the model achieve its highest potential, without causing the model to overfit/underfit and worsen the test scores. For a clear study we decided to have the same original model while only changing the epochs studied by a range of 1 to 200, annotating both the test and train F1 scores. Looking at Fig.5 we can see an initial exponential rise on the first 20 epochs on both the train and test. The rest of the epochs evaluated as expected showed an increase in train eventually reaching 100% F1-Score while the test reached a cap at 90%. Another notable point was the increase in difference between test and train scores, at first the scores at the same epoch number were nearly identical, while at higher epochs the difference was larger.

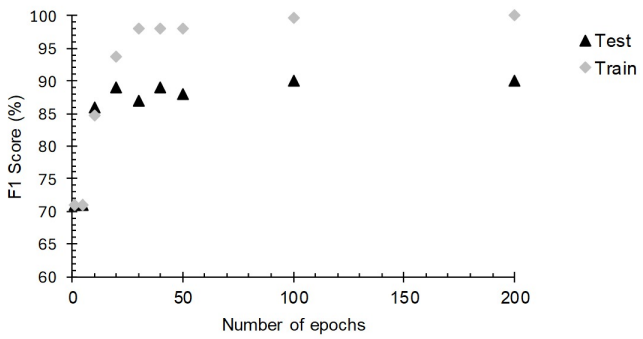

Figure 5: F1 Score variation according to the number of epochs in Original dataset

The same test was done with an increased dataset, this showed similar results but now with a higher difference in between train and tests and a lower F1-Score peak at 87%. In order to reach a more precise epoch number and avoid overfitting we created a code that conducts an early stopping. The code worked by creating a validation dataset, annotating their scores and stopping the learning process when the validation drops by a number monitored through the patience parameter (80% for training and 20% for validation). The early stop method was later added to the final assessments performed with other hyperparameters to the original and added dataset without any new pre-trained model.

2.2.3 *Learning rate.*

The learning rate is a hyperparameter that can have a significant effect on the final evaluation metric values. Too fast a learning rate can cause an unstable training process eventually causing a reliance in an unwanted set of weights, while a slower rate can cause the learning to halt being unable to proceed. For this hyperparameter we studied 4 different rates: 1e-1,1e-2,1e-3,1e-4. After examining models with different hyperparameters with each rate we saw the same tendencies. Models with a rate of 1e-1 had considerably different F1-scores for each language variants and the lowest weighted averages F1-Scores of all the rates considered (37%), not having enough time to let the train learn. The learning rates 1e-2 and 1e-3 contained the more stable results, having almost identical F1-Scores between classifications and generally higher results, although 1e-2 did not have a model with an average higher than 77% while 1e-3 contained the highest results reaching F1-scores of 91%. The learning rate 1e-4 also had considerably low results, the rate being too slow to properly train the model.

2.2.4 *Batch number.*

The batch size was fixed to 32[2] with no considerable difference observed for 16 and 64.

2.2.5 *Optimization function.*

A speed boost comes from using faster optimization than the regular Gradient Descent (GD). The AdaGrad, RMSProp, the Adam and AdamW optimization functions were tried with the last presenting better results for a fixed number of 20 epochs.

Table 2: Optimization function evaluation Dataset original with Epochs =20, hidden_sizes=[200,200, 200, 200]

Optimizer	Test F1 pt-pt	Test F1 pt-br	Time (s)
RMSprop	0.889852	0.871362	630.883897
Adagrad	0.901034	0.889803	628.558668
Adam	0.892415	0.891236	629.294549
AdamW	0.914158	0.905676	631.804818

2.2.6 *Number of hidden layers and neurons.*

Usually, in more complex models' deeper networks tend to have a better performance. To experiment with this parameter, we modified the number of layers and the hidden size to understand how these changes affect the metric scores. For this point we made a number of experiments interchanging either the number of layers or hidden size. At first these experiments returned unreliable and unstable results, but a more comprehensive approach made us have a better understanding on how the layers and hidden size effected the scores. We first tracked the performance of the model through the increase of layers with each of them containing the same amount of hidden size (128), the model was also under the early stopping method. We observed a slight increase in test scores in proportion to the number of layers eventually reaching a plateau at 5 layers. This helped us believe that 4 layers were the optimum amount in order to get a balance between model complexity and a high F1-score. Fixing the number of layers at 4 (Figure 6), we uniformly increased the size of the hidden layer and the result converged to 89.1% with a total number of 1000 neurons (4 layers with a size of 250). Next, we used the extended dataset (including *journalistc* and *politics*) and the F1 score benefited up to a maximum value of 92.17% for a total number of neurons of 1250 (4 layers with a size of 300). Finally, we increased the size of the hidden layer and the total number of layers with the extended dataset which showed no benefit.

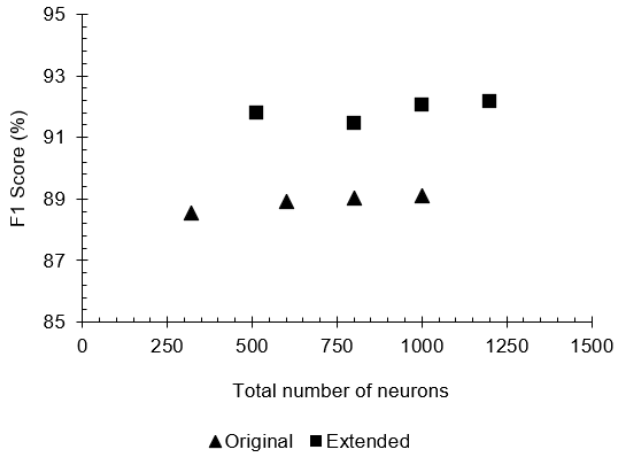

Figure 6: F1 Score variation according to the total number of neurons for the original and extended dataset

2.3 Pre-trained components

2.3.1 Word embeddings.

Other possible improvements on the model, would incorporate the use of pre-trained word embeddings on the code. The embedding layer creates a dense vector representation of a word in a vocabulary of the input data [2]. This representation will make it easier for the neural network to make accurate predictions [2]. So to improve the embedding layer, we replaced the original one with pre-trained embedding. We experimented with two libraries, GLOVE [7], and fastText [1]. For this experiment, we used 4 layers. We tested both embeddings using the original dataset and the extended dataset. Below is the table with the results for both libraries (Table 3). The model showed better results with the extended dataset. Regarding the pre-trained word embeddings using the original dataset, we observed better performance with GloVe.

Table 3: Performance of the Word Embeddings

	Original		Extended	
Word Embedding	**Glove**	**Fasttext**	**Glove**	**Fasttext**
Test F1 pt-pt	0.8958	0.8579	0.9125	0.9148
Test F1 pt-br	0.8867	0.8565	0.9043	0.9041
Epochs	36	36	36	36

2.3.2 Pre-trained model.

To improve the model, we tested a pre-trained model. The Albertina model [8] had installation issues, so we did not use it. The next pre-trained model tested was BERTimbau, using its base model. The specific tokenize was used and it included the attention mask. Taking into consideration the attention mask, in the forward function, after getting the input_ids we gave a value of 1 if the input does not correspond to the pad_token_id and 0 if they are the same as pad_token_id [4]. Using this pre-trained model, we did not use an additional embedding layer, as this layer is already included in the BERT model [5]. Also in the forward function, after obtaining the input_ids, we created an attention mask where the tokens that corresponded to pad_token_id were given a value of 0, all other tokens had a value of 1. This ensured that padding tokens did not affect the model's computation [6].

Then, we obtained the outputs given from the hiden_state and applied a linear function for classification. The hidden size for the linear model was set to 768, which matched the output dimension of the Bertimbau model's hidden states. The optimization function used was the AdamW. The learning rate was 1e-03. We also implemented early stopping with a patience of 2, due to the time this model took in each epoch. This was done to prevent overfitting and to save computational resources. We tested this model with the original dataset and with the extended dataset. Training the model with the original dataset gave F1 scores of 0.9759 and 0.9742 for PTPT and PTBR respectively. With the extended dataset the model yielded 0.9808 and 0.9795 for PTPT and PTBR respectively.

This was the model that presented the best results in our experiments, and so it was the chosen one for the submission of the results in Kagle.

3 CONCLUSIONS

The best alterations to the original model occurred with the addition of the pre-trained model Bertimbau's. This model reached test score of 98% showing us how well it differentiated the language variants Portuguese (Portugal) from Portuguese (Brasil). For future projects, we believe that a more comprehensive approach to the experiments could be made by either testing a bigger range of hyperparameters or other pre-trained models. Another addition, that could benefit this project would be performing the tests locally with a better performing computer or subscribing to a feature in google collab that enables unlimited GPU access, since the time management and GPU block by google collab was one of our biggest obstacles.

REFERENCES

[1] Piotr Bojanowski, Edouard Grave, Armand Joulin, and Tomas Mikolov. 2016. Enriching Word Vectors with Subword Information. *arXiv preprint arXiv:1607.04606* (2016).

[2] Aurélien Géron. 2019. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow: Concepts, Tools, and Techniques to Build Intelligent Systems* (2nd ed.). O'Reilly Media, Sebastopol, CA. https://www.oreilly.com/library/view/hands-on-machine-learning/9781492032632/

[3] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. 2009. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer, New York, NY. https://doi.org/10.1007/978-0-387-84858-7

[4] huggingface. 2020. *Clarification on the attention_mask.* Retrieved May 29, 2024 from https://discuss.huggingface.co/t/clarification-on-the-attention-mask/1538

[5] Init. 2020. Why BERT has 3 Embedding Layers and Their Implementation Details. https://medium.com/@_init_/why-bert-has-3-embedding-layers-and-their-implementation-details-9c261108e28a. Accessed: 2024-06-02.

[6] Ganesh Lokare. 2023. *Preparing Text Data for Transformers: Tokenization, Mapping and Padding.* Retrieved May 29, 2024 from https://medium.com/@lokaregns/preparing-text-data-for-transformers-tokenization-mapping-and-padding-9fbfbce28028

[7] Jeffrey Pennington, Richard Socher, and Christopher D. Manning. 2014. GloVe: Global Vectors for Word Representation. In *Empirical Methods in Natural Language Processing (EMNLP)*. 1532–1543. http://www.aclweb.org/anthology/D14-1162

[8] Rodrigo Santos, João Rodrigues, Luís Gomes, João Silva, António Branco, Henrique Lopes Cardoso, Tomás Freitas Osório, and Bernardo Leite. 2024. Fostering the Ecosystem of Open Neural Encoders for Portuguese with Albertina PT-* family. arXiv:2403.01897 [cs.CL]